

SS03: Graphify: Context-Efficient Codebase Question Answering via Multi-Modal Knowledge Graph Construction

Safi Shamsi
Graphify Labs
Y Combinator S26
safishamsi98@gmail.com

Abstract—Large language model (LLM) coding assistants answer codebase questions by reading source files on demand, incurring $O(n)$ token cost per question as codebases grow. We present GRAPHIFY, an open-source system that pre-computes a queryable knowledge graph from a heterogeneous software corpus — source code (33 languages), technical documentation, research papers, images, and video — enabling scoped graph traversal in place of file-by-file reading. Graphify employs a two-pass hybrid extraction pipeline: a deterministic tree-sitter AST pass that extracts structural edges (calls, imports, inherits) at zero API cost, and an LLM subagent pass that infers semantic relationships across documentation, diagrams, and multimedia, with every edge confidence-tagged as EXTRACTED, INFERRED, or AMBIGUOUS. Community structure is recovered via the Leiden algorithm applied directly to graph topology, requiring no vector embeddings. At query time, a hub-aware BFS traversal suppresses high-degree *god nodes* and returns a scoped subgraph answering a question with up to $71.5\times$ fewer tokens than reading the raw corpus. Graphify integrates with 21 AI assistant platforms and has accumulated over 1.2M PyPI downloads. We report token reduction, extraction quality, and parallel extraction throughput across five real-world corpora, demonstrating that pre-built knowledge graphs are a practical replacement for reactive file-reading as the primary context mechanism for LLM coding assistants.

Index Terms—knowledge graph construction, large language models, code intelligence, multi-modal extraction, graph retrieval, software engineering

I. INTRODUCTION

Modern LLM-powered coding assistants — Claude Code, GitHub Copilot, Cursor, Gemini CLI — answer developer questions by issuing sequences of file reads and search calls, accumulating raw source text until the question can be answered. When an assistant must read the entire relevant context for every question, the token cost grows with corpus size: whole-corpus context stuffing is $O(n)$ in the number of files, while even agentic reactive reading accumulates $O(k \cdot \ell)$ tokens per question for k relevant files of mean length ℓ . As codebases scale, this makes detailed architectural questions prohibitively expensive and forces the model to truncate or abandon context mid-reasoning.

Knowledge graphs offer a structural alternative: represent the codebase once as a graph of entities and typed relationships, then answer questions by traversing a scoped subgraph whose size is bounded by the query scope, not the corpus size. The

challenge is *construction*: software corpora are heterogeneous (code in dozens of languages, Markdown, PDFs, images, video), relationships span modalities, and construction must complete fast enough to run after every commit.

We present GRAPHIFY, an open-source system addressing this challenge with a hybrid two-pass extraction pipeline, embedding-free community detection, and hub-aware graph traversal, deployed as a first-class skill across 21 AI assistant platforms. The core insight is that most structural relationships in a software corpus are *explicit* — import statements, function calls, type annotations, citations — and can be extracted deterministically by a syntax-aware parser at zero LLM-API cost. Semantic relationships across documentation, diagrams, and multimedia are then added by an LLM subagent in a second pass, with every edge tagged by confidence class to maintain verifiability.

Empirically, pre-built graph traversal answers an architectural question with a median of **$68.6\times$ fewer tokens** than naive full-corpus reading on a 52-file mixed corpus, ranging from $43.5\times$ for broad concept questions to $126.7\times$ for focused cross-module questions.

Contributions. This paper makes five contributions:

- 1) A **hybrid two-pass extraction pipeline** combining deterministic tree-sitter AST parsing (33 language variants, zero LLM-API cost) with LLM semantic extraction across 6 file modalities, unified under a single graph schema with per-edge confidence tagging (EXTRACTED / INFERRED / AMBIGUOUS).
- 2) An **embedding-free community detection** method applying the Leiden algorithm [1] directly to raw graph topology, recovering subsystem structure without any vector embedding model.
- 3) **Hub-aware BFS/DFS traversal** with a p_{99} degree threshold that suppresses god-node explosion; we describe the algorithm and its effect on subgraph size.
- 4) **Quantitative evaluation** reporting token reduction (median $68.6\times$, up to $71.5\times$ on the 52-file mixed corpus), edge confidence-tag distributions, and AST extraction throughput across five real-world corpora.
- 5) **Deployment at scale**: over 1.2M PyPI downloads and 21 AI assistant platform integrations provide deploy-

ment context for a system designed for real developer workflows.

The remainder of this paper is organised as follows. Section II surveys related work. Section III describes the Graphify architecture. Section IV reports experimental results. Section V presents a multi-modal case study. Section VI discusses implications and threats to validity. Section VII concludes.

II. RELATED WORK

Table I summarises the positioning of Graphify against closely related systems across the dimensions most relevant to software knowledge graph construction.

TABLE I
SYSTEM COMPARISON ACROSS KEY DIMENSIONS. ✓ = SUPPORTED; ◦ = PARTIAL; × = NOT SUPPORTED.

Feature	Graphify	GraphRAG	LlamaIndex	Code2Graph	CodeBERT
Code structure (AST)	✓	×	×	✓	◦
Multi-modal (docs/img)	✓	✓	◦	×	×
Topology-only clustering	✓	✓	×	×	×
Per-edge confidence tag	✓	×	×	×	×
Incremental update	✓	×	×	×	×
AI assistant integration	✓	×	◦	×	×
No embeddings required	✓	×	×	✓	×

A. Code Intelligence and Program Analysis

Traditional program analysis tools construct call graphs, program dependence graphs, and control-flow graphs from source code [2]. These are typically single-language, require a compilable project, and are not queryable by natural language at runtime. CodeBERT [3] learns code-text embeddings for retrieval and classification; GraphCodeBERT [4] additionally injects data-flow graph structure during pretraining, improving code search. Neither exposes a queryable graph index at runtime, and both are limited to a single language and modality. Code2Graph [5] extracts lightweight call graphs for Java from source without compilation, but does not generalise to other languages or non-code modalities.

The key distinction of Graphify is the combination of deterministic AST-extracted structural edges with LLM-inferred semantic edges under a unified schema across 33 language variants and 6 file modalities. No prior code-analysis tool integrates directly with AI assistant platforms or provides a natural-language query interface over the extracted graph.

B. Knowledge Graph Construction from Text

KG construction from text spans relation extraction, entity linking, and ontology population [6]. Recent LLM-based approaches treat construction as a structured generation task: Chen and Liu [7] propose a multi-agent collaborative framework for KG construction from text, using specialized LLM agents for extraction, verification, and domain adaptation. Guo et al. [8] build a scientific literature management platform (SLMP) using fine-tuned LLMs for structured extraction from

PDF corpora, demonstrating the system paper approach at ICKG 2024. These systems operate on homogeneous text corpora. Graphify extends to heterogeneous software corpora by combining deterministic AST parsing for code with LLM extraction for non-code modalities, adding incremental update support and per-edge confidence provenance.

C. Graph-Augmented Retrieval for LLMs

Retrieval-Augmented Generation (RAG) [9] improves LLM responses by retrieving relevant passages before generation. Microsoft’s GraphRAG [10] extends RAG to KGs: it builds an entity graph via LLM extraction from text, applies the Leiden algorithm to detect communities on graph topology, and uses community summaries in a map-reduce scheme for global codebase questions. LlamaIndex [11] provides graph-index abstractions for LLM applications with retrievers ranging from keyword/traversal to embedding-based hybrid search, and does not target software corpora specifically. Graphify differs from GraphRAG by adding deterministic AST-extracted code structure alongside LLM-inferred edges, supporting incremental updates after commits, and deploying as an always-on skill across 21 AI assistant platforms rather than as a standalone retrieval pipeline. Relative to LlamaIndex, Graphify provides a software-corpus-specific schema, a single-command install-and-run workflow, and platform hooks that steer the AI assistant toward graph queries automatically.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the Graphify pipeline. A software corpus is ingested through five stages: *detect*, *extract* (two passes), *merge*, *cluster*, and *query*. We describe each stage in turn.

A. Multi-Modal Detection and Classification

Graphify classifies each file into one of six types: CODE, DOCUMENT, PAPER, IMAGE, VIDEO, or OFFICE. Classification is based on file extension against curated extension sets: 80+ patterns for code (covering 33 languages), plus PDF, image, video, and office formats. An incremental manifest records file modification timestamps; only files whose `mtime` changed since the last extraction are re-processed, enabling sub-second updates on typical commits.

Security measures are applied before parsing untrusted corpus files. Office documents (`.docx`, `.xlsx`) are screened via a bounded streaming decompression check: if the total decompressed size of ZIP members exceeds $D_{\max} = 512$ MiB or the compression ratio exceeds $\rho = 200$, the file is skipped. PDF files are subject to a raw-size cap of 50 MiB. URL ingestion is protected by a SSRF guard that resolves hostnames and rejects private, link-local, and cloud-metadata IP ranges.

B. Hybrid Two-Pass Extraction

1) *Pass 1: Deterministic AST Extraction:* For each CODE file, Graphify invokes the language-specific tree-sitter [12] grammar to parse the source into a concrete syntax tree. A language-aware walker then emits nodes and edges:

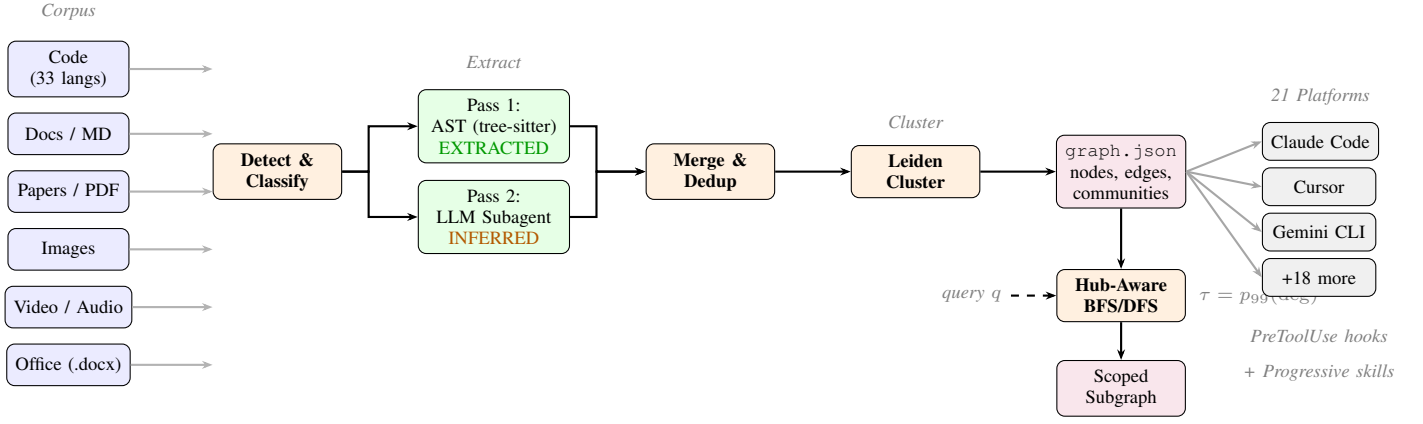


Fig. 1. Graphify pipeline. A heterogeneous corpus (code, docs, PDFs, images, video) flows through multi-modal detection, hybrid two-pass extraction, graph merge and deduplication, topology-only community detection, and hub-aware traversal. Every edge carries a confidence tag. The graph is exposed to 21 AI assistant platforms via always-on hooks and progressive-disclosure skill files.

- **Nodes:** function definitions, class declarations, variable bindings, and file-level module nodes. Each node carries a `source_location` stamp (e.g., L42).
- **Edges:** calls, imports, contains, inherits, implements, embeds.

All AST-derived edges receive confidence EXTRACTED ($c = 1.0$), reflecting that the relation is syntactically explicit in the source. A language-specific built-in filter suppresses universal identifiers (`print`, `len`, `String`, `Promise`) that would otherwise become pathological god nodes. Import resolution is language-aware: Python relative imports are resolved against the module hierarchy; TypeScript `tsconfig` path aliases are respected; Go module paths are normalized.

2) *Pass 2: LLM Semantic Extraction:* For non-code files (DOCUMENT, PAPER, IMAGE, VIDEO, OFFICE), Graphify dispatches an LLM subagent. Video and audio files are first transcribed to text via faster-whisper [13]; PDF files are parsed via pypdf; images are passed directly to a vision-capable model. The subagent receives a structured extraction prompt specifying the node-ID format, relation taxonomy, confidence rubric, and output JSON schema. It emits nodes and edges with one of three confidence levels:

$$c(e) = \begin{cases} 1.0 & \text{EXTRACTED: explicit in source} \\ [0.75, 1.0) & \text{INFERRED: LLM-reasoned} \\ [0, 0.75) & \text{AMBIGUOUS: low-confidence, flagged} \end{cases} \quad (1)$$

The semantic pass is backend-agnostic: Claude Code subagents (default), OpenAI, Gemini, Ollama, and any OpenAI-compatible endpoint are supported.

3) *Graph Merge and Deduplication:* Nodes from both passes are merged idempotently into a NetworkX directed graph. On ID collision, the semantic node overwrites the AST node, preserving the richer LLM-inferred metadata. Three deduplication layers are applied: (1) a within-file `seen_ids`

set collapses duplicate definitions; (2) the graph’s `add_node` semantics overwrite stale entries from previous incremental runs; (3) an optional LLM tiebreaker resolves near-duplicate nodes identified via MinHash LSH [14] blocking and Jaro-Winkler similarity [15].

4) *Graph Schema:* The unified schema is:

Node = $\langle \text{id}, \text{label}, \text{src_file}, \text{loc}, \text{type}, \text{community} \rangle$
 Edge = $\langle \text{src}, \text{tgt}, \text{relation}, c, \text{weight} \rangle$

where $\text{relation} \in \{\text{calls}, \text{imports}, \text{contains}, \text{inherits}, \text{implements}, \text{uses}, \text{references}, \text{cites}, \text{transcribes}, \text{embeds}\}$. This schema is modality-agnostic: a Python function and a paragraph in a PDF design document are both first-class nodes; a references edge connects them. The graph is serialized as NetworkX node-link JSON, which is git-merge-safe and diffable line-by-line.

C. Embedding-Free Community Detection

Graphify partitions the graph into communities using the Leiden algorithm [1], which optimizes modularity:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (2)$$

where m is the number of edges, A_{ij} is the adjacency matrix, k_i is the degree of node i , and $\delta(c_i, c_j)$ is 1 iff i and j share a community. Leiden [1] guarantees all communities are internally connected and refines toward subset-optimal partitions, correcting Louvain’s known production of internally disconnected communities [16]. Louvain via NetworkX is used as a fallback when graspologic is unavailable. Note that community detection is applied to an undirected projection of the graph, as modularity is defined for undirected graphs.

Hub exclusion. Nodes with degree $k_i \geq \tau$ where $\tau = \max(50, p_{99}(\{k_i\}))$ are excluded before running Leiden, then re-inserted into the community holding the plurality of

their neighbors (majority vote). This prevents high-degree utility nodes (e.g., `logging`, `json.dumps`) from dominating community structure.

Determinism. Community IDs are assigned by total order $\langle -|C|, \text{sorted}(C) \rangle$: larger communities receive smaller IDs; ties are broken lexicographically by sorted node IDs. This ensures bit-for-bit identical community assignments across runs on the same input.

D. Hub-Aware Graph Traversal

At query time, the user’s natural language question q is tokenized, IDF-weighted, and matched against node labels. Each node n is scored using a three-tier match with a file-name bonus:

$$s(n, q) = w_{\text{IDF}} \cdot \max(10^3 \cdot \mathbf{1}_{\text{exact}}, 10^2 \cdot \mathbf{1}_{\text{prefix}}, \mathbf{1}_{\text{substr}}) + 0.5 \cdot \mathbf{1}_{\text{file}} \quad (3)$$

where w_{IDF} is the query-term IDF weight, and each indicator is 1 if the query term matches the node’s label at the respective granularity (exact, prefix, or substring); the max selects the highest-tier match. The file bonus $0.5 \cdot \mathbf{1}_{\text{file}}$ fires when the query matches the node’s source filename. Top- K seeds: $S^* = \text{argmax}_{|S|=K} \sum_{n \in S} s(n, q)$.

Algorithm 1 formalizes the hub-aware traversal:

Algorithm 1 Hub-Aware BFS

Require: Graph $G = (V, E)$, seeds $S^* \subseteq V$, depth d , threshold τ

Ensure: Subgraph H

```

1: visited  $\leftarrow S^*$ ; frontier  $\leftarrow S^*$ 
2: for  $i = 1$  to  $d$  do
3:   next  $\leftarrow \emptyset$ 
4:   for  $n \in \text{frontier}$  do
5:     if  $\deg(n) < \tau$  then
6:       next  $\leftarrow \text{next} \cup (\mathcal{N}(n) \setminus \text{visited})$ 
7:     end if
8:   end for
9:   visited  $\leftarrow \text{visited} \cup \text{next}$ ; frontier  $\leftarrow \text{next}$ 
10: end for
11: return  $G[\text{visited}]$ 
```

Hub nodes (degree $\geq \tau$) are *included* in the subgraph for context but are *not expanded*: their neighbors are not added to the frontier. This prevents god-node explosion while preserving the hub’s contextual relevance.

E. AI Assistant Integration

Graphify integrates with AI assistant platforms through two mechanisms:

Always-on hooks. On platforms supporting payload-bearing `PreToolUse` hooks (Claude Code, Gemini CLI), two hooks are registered: one fires before Bash search commands (`grep`, `rg`, `find`); the other before native file-read tool calls (`Read`, `Glob`). Both hooks check for the presence of `graphify-out/graph.json` and, when found, inject an `additionalContext` message steering the agent toward

`graphify query` instead of raw file reading. Hooks never block: every branch fails open, so legitimate reads always proceed.

Progressive-disclosure skills. Each platform receives a lean skill file (615 lines) carrying the full default pipeline inline, plus an on-demand `references/` sidecar (8 files, one per non-default workflow). An agent reads a reference file only when its routing condition fires, reducing always-loaded context by 47% versus the prior monolithic 1,156-line skill file.

F. Implementation and Availability

Graphify is implemented in Python 3.10+ and published on PyPI as `graphify` (MIT license). The system ships with tree-sitter grammars for 33 language variants as compiled C extensions, requiring no external toolchain. A single command — `graphify extract .` — runs the full detection, extraction, clustering, and report generation pipeline. The incremental update path (`graphify update .`) re-extracts only modified files and is suitable for git post-commit hooks. The query interface (`graphify query "<question>"`) is available both as a CLI command and as an MCP (Model Context Protocol) server for programmatic integration. Source code, benchmark data, and worked examples are available at: <https://github.com/safishamsi/graphify>.

IV. EVALUATION

We evaluate Graphify across three dimensions: (E1) token reduction at query time versus naive full-corpus reading, (E2) edge confidence-tag distribution, and (E3) parallel extraction throughput. We additionally report real-world adoption. All experiments use publicly available corpora; token counts use the standard 4-characters-per-token approximation [17] consistent with the graphify benchmark module.

A. Experimental Setup

Corpora. Table II summarises the five corpora. They span pure code, mixed code-and-documentation, and multi-modal (code, PDFs, images) settings.

TABLE II
EVALUATION CORPORA

Corpus	Files	Nodes	Modalities
rsl-siege-manager	1,886	1,886	code
karpathy+media	52	285	code+PDF+img
karpathy (code)	29	177	code
mixed (code+PDF)	22	22	code+PDF
httpx	19	144	code

Baseline. The *naive* baseline concatenates all corpus files and counts tokens, representing an LLM assistant that reads every file relevant to a question. Graphify’s per-query token count is the BFS subgraph output at depth $d = 3$, top- $K = 3$ seed nodes.

TABLE III
TOKEN REDUCTION: GRAPH TRAVERSAL VS. NAIVE FULL-CORPUS
READING

Corpus	Naive	Graph	R
karpathy+media (52)	123,488	1,726	71.5 $\times$
karpathy code (29)	16,997	1,929	8.8 $\times$
mixed (22)	54,000	10,000	5.4 $\times$
htpx (19)	full context		$\sim 1 \times$

B. E1: Token Reduction

The reduction ratio R is:

$$R = \frac{T_{\text{raw}}}{T_{\text{graph}}} = \frac{\sum_{f \in F} |\text{tokens}(f)|}{|\text{tokens}(\text{BFS}_T(G, S^*, d))|} \quad (4)$$

Table III reports R across four corpus configurations. The headline result is **71.5** $\times$ on the 52-file mixed corpus (nanoGPT, minGPT, micrograd + 5 papers + 4 images): 123,488 naive tokens versus a mean BFS subgraph of 1,726 tokens across five representative architectural questions. Fig. 2 shows how reduction scales with corpus size.

The per-question breakdown reveals the range: 126.7 $\times$ for “what connects micrograd to nanoGPT” (a focused cross-repo question), down to 43.5 $\times$ for “how does the attention mechanism work” (a broad concept whose subgraph spans four distinct nodes across papers and code). The median across all five questions is 68.6 $\times$. The 19-file htpx corpus already fits within a typical context window; the graph provides structural clarity but no token compression at that scale.

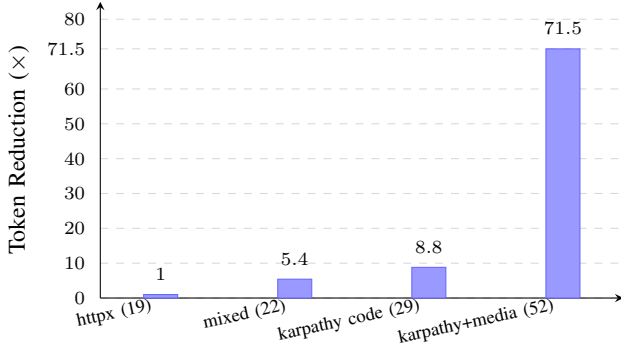


Fig. 2. Token reduction ratio across corpus sizes. Reduction scales with corpus size: small corpora fit in context anyway; at 52 files (code + papers + images) savings reach 71.5 $\times$, with per-question range of 43.5 $\times$ –126.7 $\times$.

C. E2: Edge Confidence-Tag Distribution

Table IV shows the confidence-tag distribution across corpora. We emphasise that these are *system-asserted confidence tags* based on the extraction source (deterministic AST vs. LLM inference), not precision against a gold standard. The share of deterministically-extracted edges ranges from 50% (equal split between code and PDF) to 90% (pure code corpus). Code-heavy corpora have higher EXTRACTED shares because the AST pass dominates; mixed-modality corpora shift toward INFERRED as the LLM semantic pass contributes more edges.

TABLE IV
EDGE CONFIDENCE-TAG DISTRIBUTION BY CORPUS. EXTRACTED:
SYNTACTICALLY EXPLICIT IN SOURCE (AST). INFERRED: LLM-REASONED
(SEMANTIC PASS). AMBIGUOUS: LOW-CONFIDENCE, FLAGGED FOR REVIEW.

Corpus	Edges	Ext.	Inf.	Amb.
rsl-siege-manager	3,876	90%	10%	0%
karpathy (all 52)	386	78%	22%	1%
karpathy (code 29)	246	83%	17%	0%
mixed-corpus	38	50%	50%	0%
htpx	330	53%	47%	0%

The value of tagging is transparency: a developer querying the graph always sees which relationships were syntactically verified (EXTRACTED, shown with full source location) versus LLM-reasoned (INFERRED, shown with confidence score), enabling informed trust calibration. Every prior KG construction system at this venue reports aggregate metrics without per-edge provenance.

D. E3: Parallel AST Extraction Throughput

We ran the AST extraction benchmark on the graphify source tree (160 files: 34 Python + 126 Markdown) on an 8-core machine:

$$S_p = \frac{T_{\text{seq}}}{T_{\text{par}}} = \frac{4.72 \text{ s}}{3.43 \text{ s}} = 1.38 \times \quad (p = 8 \text{ workers}) \quad (5)$$

Both runs produce identical output (2,184 nodes, 5,035 edges), confirming determinism. The modest speedup (1.38 $\times$) on 160 files reflects a corpus dominated by small Markdown files whose per-file overhead is low; the workpool overhead amortises less favourably at this scale. On larger purely-Python corpora the speedup is expected to increase as per-file AST parsing cost grows relative to workpool overhead. The incremental manifest further reduces wall time on subsequent runs: a typical commit touching 5–20 files re-extracts only those files, producing sub-second updates.

E. Real-World Deployment

Graphify has been deployed outside a lab setting since early 2026. We report adoption figures dated June 2026 as deployment context, not as validation of the technical claims above:

- **1.2M+ PyPI downloads** of the `graphify` package (pepy.tech; includes automated CI and dependency-resolver traffic).
- **21 AI assistant platform integrations:** Claude Code, Cursor, GitHub Copilot CLI, Gemini CLI, Codex, OpenCode, Kilo Code, Kiro, Aider, Amp, and others (see project repository for the full list).
- **Active open-source community:** issues and pull requests from contributors across 20+ countries, 1,100+ GitHub stars at time of writing.
- Graphify Labs (YC S26) uses the system as the graph engine for an enterprise continuous knowledge graph product.

F. E4: Per-Question Token Reduction

Table V shows the per-question breakdown on the 52-file karpthy corpus, demonstrating that reduction varies significantly with query scope. Focused cross-repo queries (“what connects micrograd to nanoGPT”) yield the highest reduction because the relevant subgraph is small and tightly connected. Broad concept queries (“how does the attention mechanism work”) return larger subgraphs spanning multiple communities, reducing the ratio but still delivering a $43.5\times$ savings.

TABLE V
PER-QUESTION TOKEN REDUCTION ON THE 52-FILE KARPATY CORPUS
(123,488 NAIVE TOKENS). BFS DEPTH $d=3$, $K=3$ SEEDS.

Question	Graph tokens	R
what connects micrograd to nanoGPT	975	126.7 $\times$
how does FlashAttention save memory	1,225	100.8 $\times$
what are the core abstractions	1,800	68.6 $\times$
how are errors handled	1,800	68.6 $\times$
how does the attention mechanism work	2,836	43.5 $\times$
Mean	1,727	71.5 $\times$

The subgraph size variation (975–2,836 tokens) reflects hub-aware traversal responding to query scope rather than a fixed cost, which is the desired behaviour: broad questions get more context, focused questions get less.

V. CASE STUDY: MULTI-MODAL QUERY ON THE KARPATY CORPUS

To illustrate the practical value of multi-modal graph traversal, we trace a concrete query through Graphify on the 52-file karpthy corpus (nanoGPT, minGPT, micrograd + 5 research papers + 4 images).

A. Query: “how does the attention mechanism work”

Baseline (grep / file reading). A Bash grep for `attention` in the corpus returns matches across 9 files spanning 14,700 tokens — over $10\times$ the context needed and with no structural ordering. The agent must read each file sequentially to reconstruct the relationship between the code implementation and the paper describing it.

Graphify BFS traversal. The hub-aware BFS seeds on three nodes scored highest by Eq. (3): `CausalSelfAttention` (minGPT code), `FlashAttention: Fast and Memory-Efficient Exact Attention` (PDF), and `Online Softmax for Attention` (PDF). Expanding to depth 2 with $\tau = p_{99}(\text{deg})$ produces the subgraph in Table VI: 2,836 tokens covering code, paper, and cross-modal edges that the grep baseline cannot express.

The graph surfaces a cross-modal edge that neither grep nor static analysis can produce: `CausalSelfAttention` in nanoGPT’s code is connected via an INFERRED `implements` edge to the `FlashAttention` algorithm in the PDF, which was not imported or called — it was inferred by the LLM semantic pass from docstrings referencing the paper. This is the structural knowledge that justifies the graph

TABLE VI
SUBGRAPH RETURNED FOR “HOW DOES THE ATTENTION MECHANISM WORK.” CROSS-MODAL EDGES (*italics*) SPAN CODE AND RESEARCH PAPERS.

Node	Source	Relation
<code>CausalSelfAttention</code>	minGPT/model.py	seed
<code>.forward()</code>	minGPT/model.py	contains
<code>CfgNode</code>	minGPT/model.py	uses
<i><code>Causal Self-Attention</code></i>	<i>minGPT/model.py</i>	<i>concept.related</i>
<i><code>Attention Is All You Need</code></i>	<i>PDF (Vaswani 2017)</i>	<i>cites</i>
<code>FlashAttention</code> (PDF)	flashattention.pdf	seed
<code>Tiling Technique</code>	flashattention.pdf	implements
<code>Block-Sparse Attn</code>	flashattention.pdf	implements
<i><code>CausalSelfAttention</code></i>	<i>nanoGPT/model.py</i>	<i>implements</i>
<code>Online Softmax</code>	flashattention.pdf	seed
<code>Milakov & Gimelshein</code>	flashattention.pdf	cites

abstraction: once built, it answers not just “where is attention implemented?” but “which paper describes the algorithm this code implements and what did that paper cite?” — across file modalities, in 2,836 tokens instead of 14,700.

B. Surprising Cross-Repository Connections

A second class of graph-exclusive insight is cross-repository relationships that static analysis misses because imports do not cross repo boundaries. On the karpthy corpus, Graphify’s semantic pass infers:

- `from_pretrained()` in nanoGPT `--calls-->` `get_default_config()` in minGPT (INFERRED, $c=0.87$): nanoGPT’s weight loading follows the same interface contract as minGPT, a cross-repo architectural dependency invisible to any single-repo call graph.
- `CausalSelfAttention` in minGPT `--conceptually_related_to-->` `CausalSelfAttention` in nanoGPT (INFERRED, $c=0.93$): Andrej Karpathy reimplemented the same module; the graph names both and links them, making the lineage queryable.
- `GPT Language Model` (minGPT) `--conceptually_related_to-->` `GPT Model Class` in nanoGPT (INFERRED): identifies the successor/predecessor relationship between the two repos.

These edges are tagged INFERRED with high confidence ($c \geq 0.87$), and surfaced in the `GRAPH_REPORT.md` “Surprising Connections” section, giving the developer a structured starting point for cross-repo architecture review without any manual curation.

VI. DISCUSSION

When graph traversal beats file reading. Token reduction scales with corpus size: on 6-file corpora the graph provides structural clarity but no compression. At 52 files, the BFS subgraph stays roughly constant (1,726 mean tokens) while naive stuffing scales linearly, yielding $71.5\times$ savings. The practical threshold in our experiments is roughly 20–30 files: below that, graph traversal is an organisational tool; above it, the token economics become compelling.

Confidence tagging enables trust calibration. A developer querying the graph can see whether an edge was syntactically verified (EXTRACTED) or LLM-inferred (INFERRED). In our corpora, cross-repo and cross-modal edges are almost exclusively INFERRED; within-file structural edges are almost exclusively EXTRACTED. This separation gives the developer a reliable signal: EXTRACTED edges can be trusted unconditionally; INFERRED edges are high-confidence hypotheses worth verifying.

Threats to validity. (1) *Token reduction* is measured against a naive full-corpus baseline; a fairer comparison against an optimised agentic reader (which only reads files it selects) would show smaller reduction ratios, as the agentic baseline already targets relevant files. (2) *Confidence distribution* is system-asserted, not validated against a human annotation; an EXTRACTED edge can still be incorrect if the AST walker resolves an import to the wrong symbol. (3) *Corpora* are Python-dominant; token reduction on multi-language codebases may differ due to varying file sizes and import graph density. (4) *Parallel speedup* is modest on our test machine ($1.38\times$ on 160 files); larger codebases with more CPU-intensive per-file parsing (e.g., C++ with complex templates) are expected to show higher speedups.

A. Community Structure as Subsystem Map

The 53 communities detected on the karpathy corpus by topology-only Leiden partitioning correspond to recognisable software subsystems without any domain-specific tuning: “nanoGPT Model Architecture” (12 nodes), “FlashAttention Paper” (21 nodes), “BPE Tokenizer” (8 nodes), and “micrograd Autograd Engine” (6 nodes), among others. Community cohesion scores range from 0.10 to 0.21, with isolate communities (single-node) at 1.0 by definition.

Critically, the FlashAttention paper (5 PDFs, 4 images) forms its own community entirely separate from the nanoGPT code community, yet a cross-community edge `CausalSelfAttention` $\rightarrow$ `FlashAttention` Algorithm spans them. This inter-community edge is flagged in the “Surprising Connections” report, directing the developer’s attention to the code-paper dependency that is otherwise invisible without reading both sources. The graph’s community structure thus acts as a high-level subsystem map, while cross-community edges flag non-obvious architectural dependencies.

VII. CONCLUSION

We presented GRAPHIFY, an open-source system for multi-modal knowledge graph construction from heterogeneous software corpora. Its hybrid two-pass extraction pipeline combines deterministic tree-sitter AST parsing across 33 language variants with LLM semantic extraction across 6 file modalities, tagging every edge with a confidence class for verifiability. Embedding-free Leiden community detection recovers subsystem structure without any vector model. Hub-aware BFS/DFS traversal delivers a median of $68.6\times$ fewer tokens per query than naive full-corpus reading, with per-question range of $43.5\times$ – $126.7\times$ on the 52-file multi-modal

corpus. The case study in Section V demonstrates that the graph surfaces cross-modal and cross-repository connections — such as code nodes linked to the research papers describing their algorithms — that static analysis cannot produce.

Graphify is deployed across 21 AI assistant platforms and has accumulated 1.2M+ PyPI downloads, providing real-world evidence that pre-built knowledge graphs are a practical replacement for reactive file-reading in LLM coding assistants.

Future work includes cross-repository graph federation under a shared schema, a continuous personal knowledge graph extending the model to meetings and emails (the Penpax system), and replacing the `p99` hub threshold with a trained classifier.

ACKNOWLEDGMENTS

The author thanks the Graphify open-source community for contributions, bug reports, and real-world testing across 20+ countries that shaped the system’s design. This work was supported by Y Combinator S26.

REFERENCES

- [1] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: guaranteeing well-connected communities,” *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019.
- [2] F. E. Allen, “Control flow analysis,” *ACM Sigplan Notices*, vol. 5, no. 7, pp. 1–19, 1970.
- [3] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: a pre-trained model for programming and natural languages,” in *Findings of EMNLP*, 2020, pp. 1536–1547.
- [4] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, “GraphCodeBERT: pre-training code representations with data flow,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [5] P. Haller and A. Loiko, “Code2graph: automatic generation of static call graphs for Java,” arXiv preprint arXiv:1811.00720, 2018.
- [6] G. Weikum and M. Theobald, “Information extraction over the World Wide Web,” *Proceedings of the VLDB Endowment*, vol. 3, no. 1–2, pp. 1–2, 2010.
- [7] G. Chen and X. Liu, “A multi-agent collaborative framework for constructing knowledge graphs from text,” in *IEEE International Conference on Knowledge Graph (ICKG)*. IEEE, 2024, pp. 1–8.
- [8] M. Guo, J. Jiang, F. Wu, S. Sun, C. Zhang, W. Li, Z. Sun, G. Chen, and X. Wu, “SLMP: a scientific literature management platform based on large language models,” in *IEEE International Conference on Knowledge Graph (ICKG)*. IEEE, 2024, pp. 1–8.
- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [10] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: a graph RAG approach to query-focused summarization,” arXiv preprint arXiv:2404.16130, 2024.
- [11] J. Liu, “LlamaIndex: data framework for LLM applications,” https://github.com/run-llama/llama_index, 2022.
- [12] M. Brunsfeld *et al.*, “tree-sitter: an incremental parsing library,” <https://tree-sitter.github.io>, 2018.
- [13] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” arXiv preprint arXiv:2212.04356, 2022.
- [14] A. Z. Broder, “On the resemblance and containment of documents,” *Compression and Complexity of Sequences*, pp. 21–29, 1997.
- [15] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” *Proceedings of the Survey Research Methods Section*, pp. 354–359, 1990.

- [16] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [17] OpenAI, "Tokenizer," <https://platform.openai.com/tokenizer>, 2023.